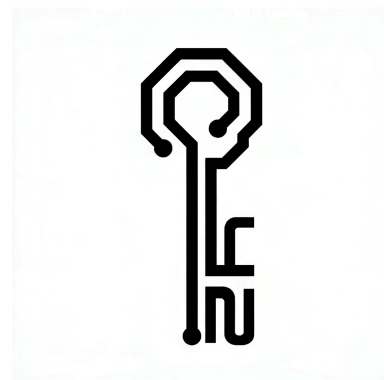


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 1
«Выделение ресурса параллелизма. Технология OpenMP»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Анализ алгоритма	3
3.1. Описание	3
3.2. Оценка	3
3.3. Блок-схема	4
3.4. Директивы OpenMP	5
4. Результаты экспериментов	6
4.1. Последовательный алгоритм	6
4.2. Параллельный алгоритм	8
5. Заключение	10
6. Приложение	11
6.1. Программные коды	11
6.2. Таблицы	15

1. Цель работы

Приобрести базовые навыки теоретического и экспериментального анализа высокопроизводительных параллельных алгоритмов, построения параллельных программ.

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (_OPENMP = 202011).

3. Анализ алгоритма

3.1. Описание

1. Создаётся массив `array` из `count` элементов
2. Массив заполняется случайными числами с использованием функции `rand()`
3. Для поиска максимального элемента используется параллельная обработка с OpenMP:
 - a. Создаётся `threads` потоков.
 - b. Каждому потоку назначается своё подмножество элементов массива.
 - c. Поток находит локальный максимум среди своих элементов.
4. После завершения работы всех потоков используется директива `reduction(max: max)`, чтобы объединить локальные максимумы в глобальный.
5. Результат выводится на экран.

3.2. Оценка

Временная сложность (при последовательном исполнении): $O(n)$.

Ускорение: $S_p = \frac{T_1}{T_p}$, где T_1 – время работы в линейном режиме (1 поток), T_p – время работы при p потоках.

Эффективность: $E_p = \frac{S_p}{p}$, где S_p – ускорение, p – количество задействованных потоков.

3.3. Блок-схема

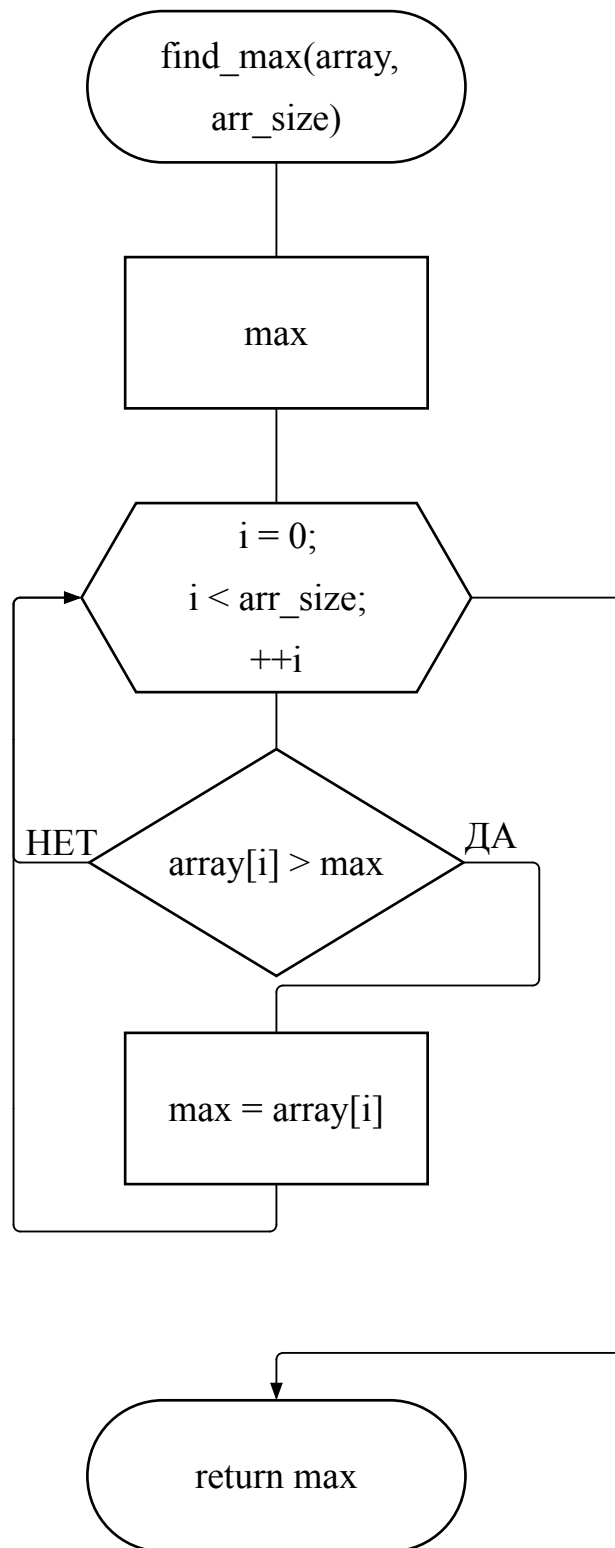


Рисунок 1 – Блок-схема алгоритма, описанного в п. 3.1

3.4. Директивы OpenMP

#pragma omp parallel num_threads(threads) shared(array, count) reduction(max: max) default(none)

- Смысл: создает параллельный регион с threads числом потоков, с заданными shared/private переменными с объединением результатов reduction.
- Область действия: блок кода { } сразу после директивы.
- Роль в программе: разбиение выполнения на несколько потоков; обеспечение корректного объединения локальных переменных через reduction.
- Без этой директивы цикл выполнялся бы последовательно одним потоком, ускорение параллельных вычислений отсутствует

#pragma omp for

- Смысл: распределение итераций цикла между потоками параллельного региона.
- Область действия: следующий цикл.
- Роль в программе: возможность одновременной параллельной обработки разных частей массива.
- Без этой директивы каждый поток выполнял бы весь цикл полностью, что привело бы к лишним вычислениям.

4. Результаты экспериментов

Каждая точка на графике была получена путем повторения эксперимента 10 раз, на одинаковых исходных данных для каждого повтора.

4.1. Последовательный алгоритм

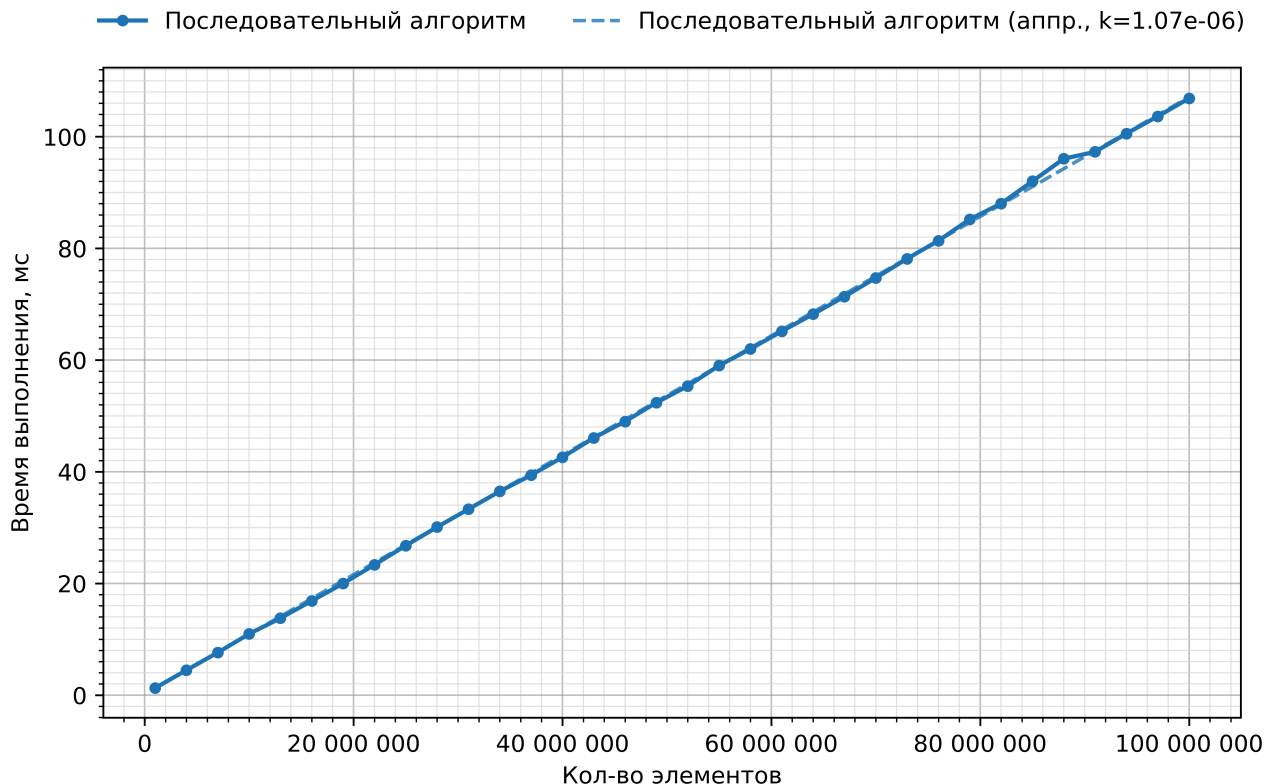


Рисунок 2 – График зависимости времени выполнения последовательного алгоритма от количества элементов в массиве

Дополнительно был построен график для диапазона 82–91 млн элементов с повышенной плотностью выборки. Каждая точка на графике соответствует среднему значению, полученному по результатам 100 повторений эксперимента.

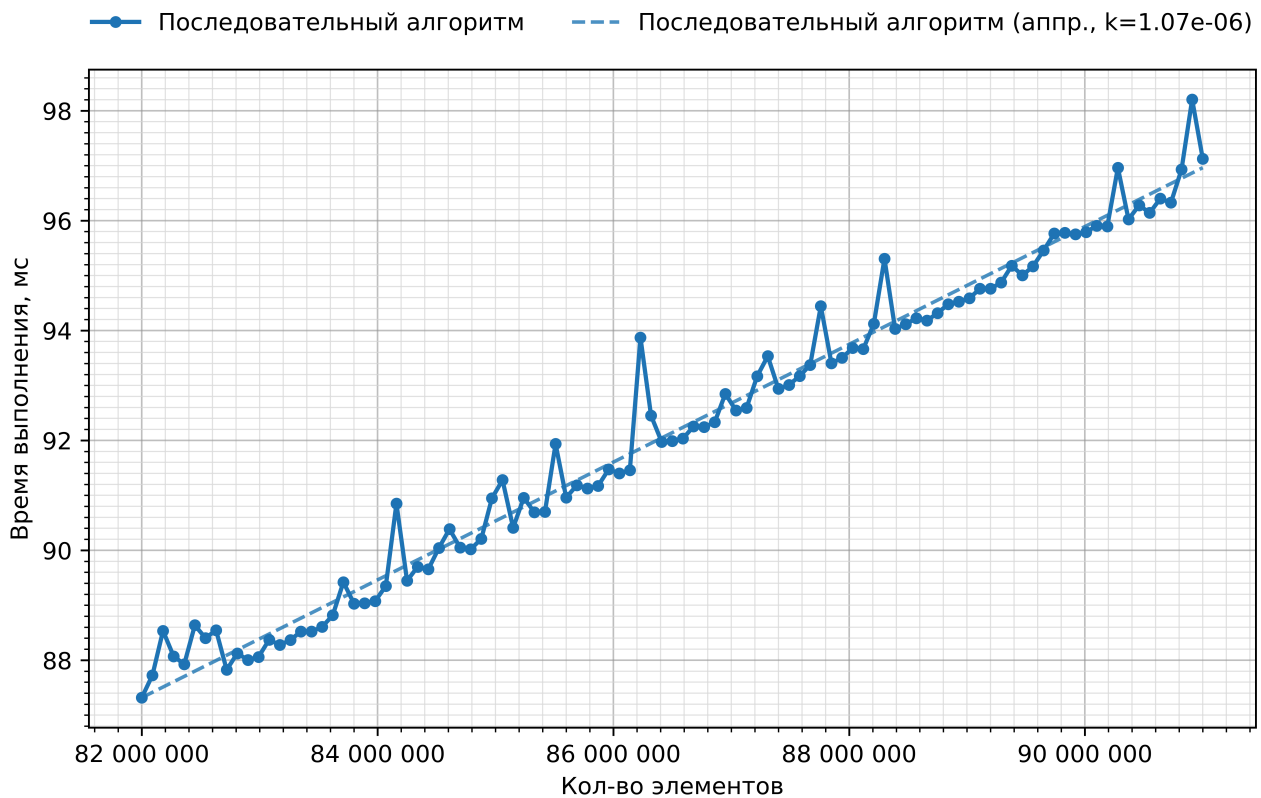


Рисунок 3 – График зависимости времени выполнения последовательного алгоритма от количества элементов в массиве для промежутка 82-91 млн. элементов

4.2. Параллельный алгоритм

Каждая точка на графике была получена путем повторения эксперимента 10 раз, на одинаковых исходных данных для каждого повтора.

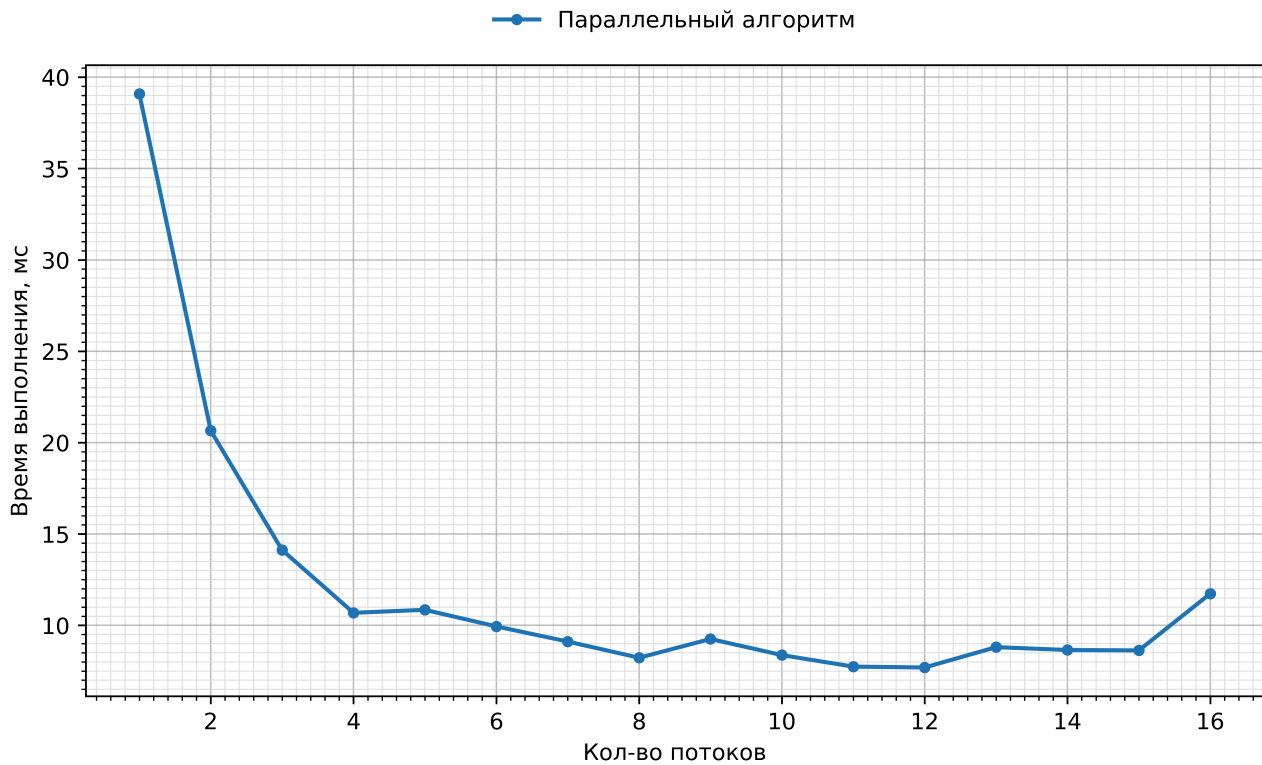


Рисунок 4 – График зависимости времени выполнения параллельного алгоритма от количества потоков (кол-во элементов: $30 \cdot 10^7$)

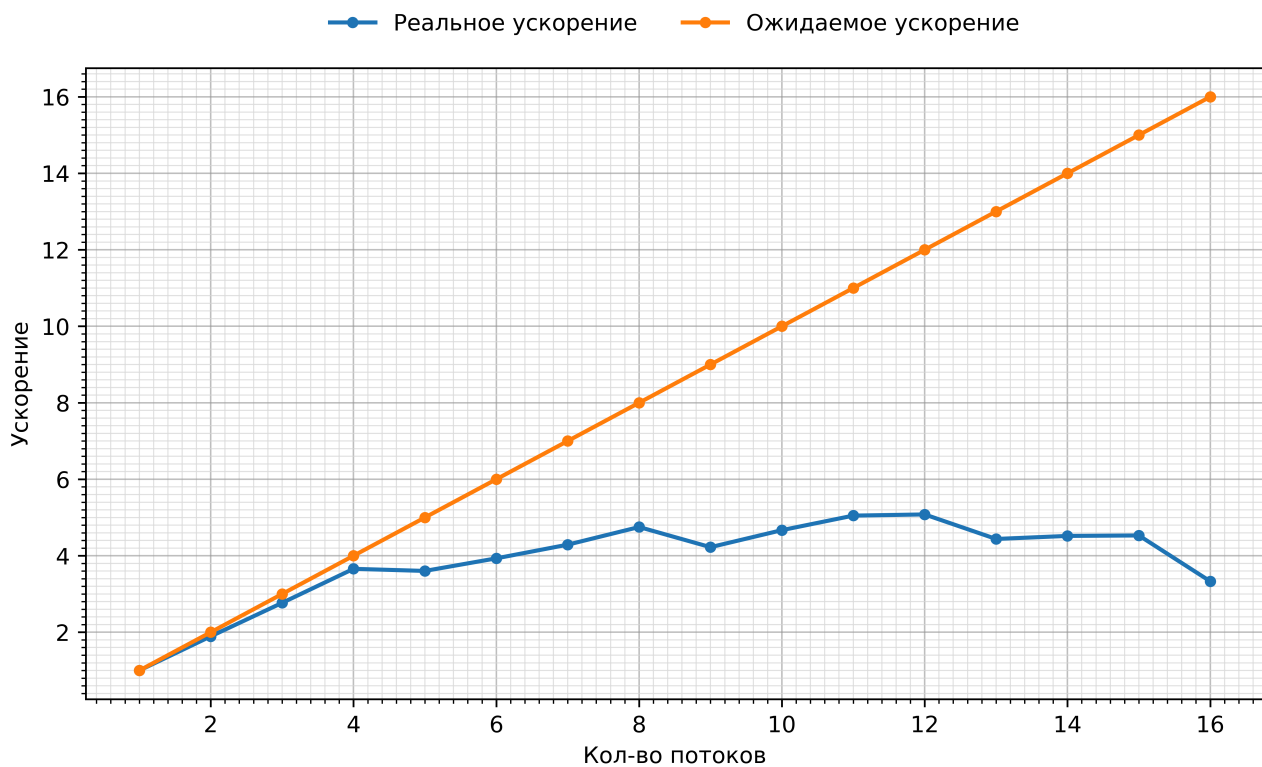


Рисунок 5 – График зависимости ускорения параллельного алгоритма от количества потоков (кол-во элементов: $30 \cdot 10^7$)

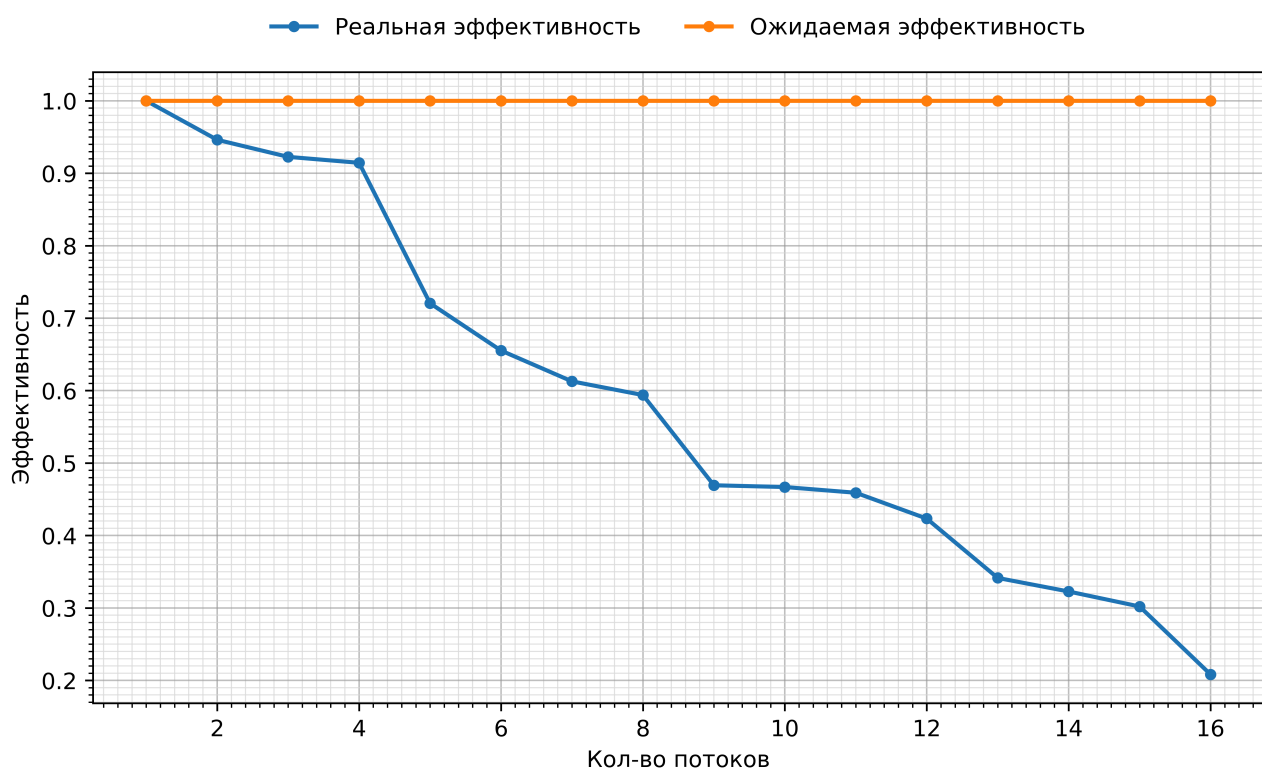


Рисунок 6 – График зависимости эффективности параллельного алгоритма от количества потоков (кол-во элементов: $30 \cdot 10^7$)

5. Заключение

В ходе выполнения лабораторной работы были освоены навыки построения параллельных алгоритмов и методы их теоретического и экспериментального анализа.

Был рассмотрен алгоритм поиска максимального элемента массива, для которого как аналитически, так и экспериментально подтверждена линейная временная сложность $O(n)$.

На основе данного алгоритма реализованы последовательная и параллельная версии программы. Проведено экспериментальное исследование ускорения и эффективности параллельных вычислений при различном числе потоков.

На графиках наблюдается заметное снижение ускорения и эффективности параллельного алгоритма при 4, 8 и 12 потоках соответственно. Это может быть связано с особенностями архитектуры процессоров Apple и управления потоками. Процессоры Apple серии M имеют определённое количество высокопроизводительных и энергоэффективных ядер. При 4 потоках, вероятно, достигается предел количества высокопроизводительных ядер, что ограничивает дальнейший линейный рост производительности. При 8 потоках задействованы все доступные ядра (включая энергоэффективные), и добавление дополнительных потоков приводит к конкуренции за ресурсы (кэш, шина данных и т. д.). После достижения 8 потоков их становится больше, чем логических процессоров, поэтому ускорение практически не растёт — система начинает активно использовать контекстные переключения, что создаёт значительные накладные расходы и снижает общую эффективность.

В ходе дополнительного исследования была проанализирована аномалия, наблюдаемая на графике зависимости времени выполнения последовательного алгоритма в диапазоне 82–91 млн элементов. Для уточнения характера данной аномалии был построен график в указанном диапазоне с повышенной плотностью выборки.

Результаты анализа показали, что выявленная аномалия носит случайный характер и обусловлена неидеальной изоляцией экспериментальной среды. В процессе проведения измерений ядра процессора одновременно выполнял посторонние процессы, что привело к локальному снижению

производительности и, как следствие, к увеличению времени выполнения программы на отдельных измерениях. На детализированном графике также отмечаются незначительные флуктуации времени выполнения, вызванные теми же внешними факторами, не связанными с особенностями исследуемого алгоритма.

6. Приложение

6.1. Программные коды

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>
#include <limits.h>

int sequential(int* array, size_t size) {
    int max = INT32_MIN;
    for(int i=0; i<size; i++)
    {
        if(array[i] > max) { max = array[i]; };
    }
    return max;
}

int main(int argc, char** argv)
{
    srand(time(NULL));

    FILE* file = fopen("results/sequential", "w");
    fprintf(file, "1\n");
    fprintf(file, "Последовательный алгоритм\n");
    fprintf(file, "Кол-во элементов\n");
    fprintf(file, "Время выполнения, мс\n");
    fprintf(file, "linear\n");
    int sz = pow(10, 6);
    int end_sz = pow(10, 8);
    int step = 3*pow(10, 6);
    int repeats = 10;

    while (sz <= end_sz) {
        int* array = (int*)malloc(sz*sizeof(int));
        for(int i=0; i<sz; i++) { array[i] = rand(); }

        double time_sum = 0;
        double start, end;
        for (int i = 0; i < repeats; ++i) {
            start = omp_get_wtime();
            sequential(array, sz);
            end = omp_get_wtime();
            time_sum += end - start;
        }
    }
}
```

```

    }
    time_sum /= 10;
    time_sum *= 1000; // в миллисекунды
    fprintf(file, "%d %lf\n", sz, time_sum);
    sz += step;
    free(array);
    printf("finished %d\n", sz);
}

fclose(file);
return 0;
}

```

Файл 1 – Замер времени обработки для последовательного алгоритма

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>
#include <limits.h>

int sequential(int* array, size_t size) {
    int max = INT32_MIN;
    for(int i=0; i<size; i++)
    {
        if(array[i] > max) { max = array[i]; };
    }
    return max;
}

int main(int argc, char** argv)
{
    srand(time(NULL));

    FILE* file = fopen("results/sequential_extra", "w");
    fprintf(file, "1\n");
    fprintf(file, "Последовательный алгоритм\n");
    fprintf(file, "Кол-во элементов\n");
    fprintf(file, "Время выполнения, мс\n");
    fprintf(file, "linear\n");
    int sz = 82*pow(10, 6);
    int end_sz = 91*pow(10, 6);
    int step = 9*pow(10, 4);
    int repeats = 100;

    while (sz <= end_sz) {
        int* array = (int*)malloc(sz*sizeof(int));
        for(int i=0; i<sz; i++) { array[i] = rand(); }

        double time_sum = 0;
        double start, end;
        for (int i = 0; i < repeats; ++i) {
            start = omp_get_wtime();
            sequential(array, sz);
            end = omp_get_wtime();

```

```

        time_sum += end - start;
    }
    time_sum /= repeats;
    time_sum *= 1000; // в миллисекунды
    fprintf(file, "%d %lf\n", sz, time_sum);
    sz += step;
    free(array);
    printf("finished %d\n", sz);
}

fclose(file);
return 0;
}

```

Файл 2 – Замер времени обработки для последовательного алгоритма на промежутке 88-92
МЛН ЭЛЕМЕНТОВ

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>
#include <limits.h>

int parallel(int* array, size_t size, size_t threads) {
    int max = INT32_MIN;
    #pragma omp parallel num_threads(threads) shared(array, size) reduction(max: max) default(none)
    {
        #pragma omp for
        for(int i=0; i<size; i++)
        {
            if(array[i] > max) { max = array[i]; };
        }
    }
    return max;
}

int main(int argc, char** argv)
{
    srand(time(NULL));

    FILE* file_res = fopen("results/parallel", "w");
    fprintf(file_res, "1\n");
    fprintf(file_res, "Параллельный алгоритм\n");
    fprintf(file_res, "Кол-во потоков\n");
    fprintf(file_res, "Время выполнения, мс\n");
    fprintf(file_res, "none\n");

    FILE* file_speedup = fopen("results/speedup", "w");
    fprintf(file_speedup, "2\n");
    fprintf(file_speedup, "Реальное ускорение\n");
    fprintf(file_speedup, "Ожидаемое ускорение\n");
    fprintf(file_speedup, "Кол-во потоков\n");
    fprintf(file_speedup, "Ускорение\n");
    fprintf(file_speedup, "none\n");
}

```

```

FILE* file_eff = fopen("results/efficiency", "w");
fprintf(file_eff, "2\n");
fprintf(file_eff, "Реальная эффективность\n");
fprintf(file_eff, "Ожидаемая эффективность\n");
fprintf(file_eff, "Кол-во потоков\n");
fprintf(file_eff, "Эффективность\n");
fprintf(file_eff, "none\n");

int sz = 3*pow(10, 7);
int repeats = 10;
int max_threads = 16;
double T1;

for (int threads=1; threads <= max_threads; ++threads){
    int* array = (int*)malloc(sz*sizeof(int));
    for(int i=0; i<sz; i++) { array[i] = rand(); }

    double time_sum = 0;
    double start, end;
    for (int i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        parallel(array, sz, threads);
        end = omp_get_wtime();
        time_sum += end - start;
    }
    time_sum /= 10;
    time_sum *= 1000; // в миллисекунды
    if (threads == 1) {
        T1 = time_sum;
    }
    double realSpeedup = T1 / time_sum;
    fprintf(file_res, "%d %lf\n", threads, time_sum);
    fprintf(file_speedup, "%d %lf %d\n", threads, realSpeedup, threads);
    fprintf(file_eff, "%d %lf %d\n", threads, realSpeedup / threads, 1);
    free(array);
    printf("finished %d\n", threads);
}

fclose(file_res);
fclose(file_speedup);
fclose(file_eff);
return 0;
}

```

Файл 3 – Замер времени, вычисление ускорения и эффективности для параллельного алгоритма

6.2. Таблицы

Таблица 1 – Сравнение теоретических и экспериментальных данных

	Теоретические значения	Экспериментальные
Ускорение	Р раз	1.93 – 4.95 раз
Эффективность	1	0.30 – 0.94

Таблица 2 – Данные о времени работы параллельного алгоритма

Кол-во потоков	Параллельный алгоритм
1	39.089799
2	20.656514
3	14.122701
4	10.687089
5	10.849500
6	9.941792
7	9.113097
8	8.227801
9	9.254003
10	8.371902
11	7.741594
12	7.694793
13	8.806515
14	8.651495
15	8.628297
16	11.734390

Таблица 3 – Данные об ускорении параллельного алгоритма

Кол-во потоков	Реальное ускорение	Ожидаемое ускорение
1	1.000000	1
2	1.892372	2
3	2.767870	3
4	3.657666	4
5	3.602913	5
6	3.931866	6
7	4.289409	7
8	4.750941	8
9	4.224096	9
10	4.669166	10
11	5.049322	11
12	5.080033	12
13	4.438737	13
14	4.518271	14
15	4.530419	15
16	3.331217	16

Таблица 4 – Данные об эффективности параллельного алгоритма

Кол-во потоков	Реальная эффективность	Ожидаемая эффективность
1	1.000000	1
2	0.946186	1
3	0.922623	1
4	0.914416	1
5	0.720583	1
6	0.655311	1
7	0.612773	1
8	0.593868	1
9	0.469344	1
10	0.466917	1
11	0.459029	1
12	0.423336	1
13	0.341441	1
14	0.322734	1
15	0.302028	1
16	0.208201	1